Supplementary Material for "PORTER: A Plan-Committed Router for Multi-Core Distributed Quantum Computers"

This supplement contains Appendices A–J of the main paper. Section, figure, and table numbers of the form "Section III-A" or "Table VII" refer to the main paper; numbers local to this document refer to the supplement itself.

Appendix A
The 80-Qubit Suite: Reconstruction Detail

The 80-qubit suite (Section IV-C of the main paper) was generated with `mqt.bench` 2.2.2 [1] (device-independent generation followed by transpilation to our basis, since the installed `mqt.bench`'s target-aware native-gate path raised an unrelated `qiskit` compatibility error on every circuit we tried). AE at this size is not directly producible by stock `mqt.bench`: its phase-estimation construction evaluates $\mathrm{Operator}(Q)^{2^j}$ for each evaluation bit j, and the accumulated floating-point non-unitarity trips `qiskit`'s unitarity check (`"Input matrix is not unitary"`) beyond roughly 27 evaluation qubits, which is why published AE files stop near 54 qubits. Since the Bernoulli operator Q is a single-qubit rotation, we instead supply the power analytically, $Q^{2^j} = R_Y(2^{j+1}\theta_p)$ (R_Y periodic in 4π), and otherwise run `mqt.bench`'s own native-gate transpilation unchanged. The CNOT sub-sequence—which alone determines routing—is decomposition-invariant in these rotation angles and is byte-identical to stock `mqt.bench` at every size the latter can build (verified at 10, 20, 25, and 28 qubits); only the routing-irrelevant single-qubit angles carry floating-point error at high bit indices.

QNN required an analogous reconstruction. Between `mqt.bench` 1.1.0 (which produced the deep 8,126-CX 64q QNN this suite uses) and the installed 2.2.2, the benchmark's definition was changed from a ZZ feature map (full all-pairs entanglement) plus a `RealAmplitudes` ansatz to a plain Z feature map (no entanglement) plus the same ansatz. The current generator therefore emits a near-linear 79-CX circuit at 80 qubits whose two-qubit interaction graph is *identical* to GHZ's — a degenerate duplicate, not a distinct benchmark. We restore the original deep QNN by rebuilding the 1.1.0 construction (ZZ feature map, `reps`=2, full entanglement; `RealAmplitudes`, `reps`=1) and transpiling it to our basis; at 64 qubits this reproduces the checked-in 1.1.0 file's CNOT sub-sequence byte-for-byte (identical ordered pair sequence, all $\binom{64}{2}$ interacting pairs, 8,126 CX), so the 80-qubit instance (a fully-connected $\binom{80}{2}$-edge interaction graph, 12,718 CX) is a faithful extrapolation of the same construction.

TABLE I
BENCHMARK CIRCUIT CX COUNTS (MEASUREMENTS/BARRIERS STRIPPED).

Circuit	25q (arch. A)	64q (arch. H)	80q (arch. H)
AE	558	1962	2780
GHZ	24	63	79
Graphstate	25	64	80
QFT	580	1966	2660
QNN	1223	8126	12718
Random	1124	1627	23381

Appendix B
External Layouts from pytket-dqc

As a third, independent layout source complementing the shared-layout protocol of Section IV-D of the main paper, we route DSABRE and PORTER from a placement generated by `pytket-dqc`'s `HypergraphPartitioning` allocator [2] (best-of-3 KaHyPar seeds on a `NISQNetwork` matching our core graph), converted to a physical layout by the same topology-aware intra-core assignment DSABRE's own pytket benchmark uses [3]: each core's assigned logical qubits are placed by descending interaction-graph degree onto physical positions ordered by ascending intra-core centrality, then locally refined by a bounded pairwise-swap pass. TELESABRE is excluded for the same structural reason as in Section IV-D (no externally supplied layout support in its CLI). `HypergraphPartitioning` only decides which *core* each logical qubit goes to; the capacity it sees for a core is the length of that core's declared qubit list in the `NISQNetwork`. Following DSABRE's stock corner-removal convention (the four lowest-degree corner qubits of the whole-chip graph; Section IV-D), we withhold those four corner qubits from the cores' declared capacity and from the later physical-placement step within each core. This guarantees only a four-qubit margin *chip-wide*, not per-core, so a core can still end up fully packed; rather than skip the run in that case, PORTER's general safeguard against fully packed cores (Section III-A) legalizes the layout automatically before routing begins. DSABRE's teledata-only mechanism does not need a free port on the side it teleports *from* and was never affected by this precondition. Because `pytket-dqc` uses gate teleportation, not state teleportation, its own reported cost is in a different unit (e-bits per Steiner-tree-amortised hyperedge, not EPR pairs per qubit move) and is included only as context, following DSABRE's own established convention for this comparison [3].

Table II reports a complete grid: PORTER's full-core safe-

TABLE II
PYTKET-MAPPING SHARED-LAYOUT COMPARISON (25Q AND 64Q SUITES), CORNERS-REMOVED NISQNETWORK CAPACITY. PYTKET E-BITS IS HYPERGRAPHPARTITIONING'S OWN COST IN A DIFFERENT UNIT (CONTEXT ONLY, NOT EPR-COMPARABLE). PORTER'S GENERAL FULL-CORE SAFEGUARD LEAVES NO MISSING ENTRIES ON THIS SUITE.

Suite	Circuit	pytket e-bits	DSABRE		PORTER	
			EPR	SWAP	EPR	SWAP
25q	ae	78	68	282	73	348
25q	ghz	3	3	17	3	17
25q	graphstate	4	4	17	4	19
25q	qft	70	91	406	118	438
25q	qnn	148	151	714	127	703
25q	random	564	431	1409	395	1340
64q	ae	264	219	1560	223	1638
64q	ghz	5	5	58	5	62
64q	graphstate	10	8	60	10	60
64q	qft	307	243	1736	278	2021
64q	qnn	621	535	5285	1031	6538
64q	random	1020	729	3770	861	4384

guard legalizes the three layouts that would otherwise violate its teleport precondition (25q QNN, 64q AE, 64q QNN), at the cost of the layout-quality hit an emergency eviction inevitably takes – most visibly on 64q QNN, where the evicted layout lands PORTER at 1,031 EPR pairs against DSABRE's 535, its worst relative result on this suite. DSABRE takes this grid: it uses fewer EPR pairs on seven of the twelve rows (25q AE/QFT, 64q AE/Graphstate/QFT/QNN/Random) to PORTER's two (25q QNN/Random), with three ties, and its geometric mean is 11% lower (63.1 vs. PORTER 70.2). Pooled with the TELESABRE-layout grid of Table III of the main paper, DSABRE uses fewer EPR pairs on twenty-three of the forty-one fixed-layout instances, PORTER on thirteen, with five ties (Section V-B).

The grid above feeds pytket-dqc's partition to PORTER as a controlled probe. The complementary pipeline question – offering the same partition to PORTER as an optional extra layout candidate beside its own, which lowers geometric-mean EPR by a further 16% on the most partitionable circuits but is kept opt-in on cost and robustness grounds – is a configuration study of PORTER itself rather than part of this comparison, and is deferred to Appendix C.

APPENDIX C
PYTKET-DQC AS AN OPT-IN PORTER LAYOUT CANDIDATE

Appendix B feeds pytket-dqc's partition to PORTER as a controlled probe; the same partition can also be offered to PORTER as one extra layout candidate beside its own (Section III-A of the main paper). Since PORTER routes far faster than pytket-dqc partitions, this is nearly free on the routing side; taking the better of the two per instance lowers the eighteen-instance geometric-mean EPR from 116.8 to 98.6 (-16%), with the gains concentrated on the most partitionable circuits (Graphstate $7\to4$, $20\to10$, $28\to11$ across the three suites; 80q AE $490\to352$). We keep it *opt-in* rather than the default for three reasons. First, on the dense and Random circuits the external partition is often *worse* for PORTER (64q QNN $864\to1{,}031$, 64q Random $737\to861$), so the best-of simply retains PORTER's own layout there. Second, the partition itself is expensive – seconds to minutes, and on dense circuits more than PORTER's entire routing pass (309 s to partition 80q Random versus $\approx$37 s to route it; 16 s versus 0.6 s on 80q AE) – against a sub-millisecond cost for PORTER's own layout build, so adopting it by default would erase PORTER's compile-time advantage (Section V-F of the main paper). Third, it reintroduces the external-partitioner dependency PORTER's self-contained layout avoids, and unlike it can leave a core fully packed, relying on the full-core repair of Section III-A. PORTER therefore defaults to its own layout and treats an external partition as an optional refinement, selected only when EPR is at a premium and its generation time is acceptable.

APPENDIX D
INDEPENDENT REPLAY VERIFICATION

Beyond code review, we built an independent replay verifier that treats the router as a black box: it takes only the emitted `CompiledOp` stream, the original circuit, and the initial layout, and re-derives, from scratch, whether the stream is a faithful and physically legal execution. The verifier maintains its own physical-position map and a per-logical-qubit FIFO queue of pending gate IDs (derived directly from the circuit, independent of any router-internal DAG state), and checks, per emitted operation:

- every gate is popped from the head of *every* one of its operand qubits' queues before being marked executed, so gates cannot execute out of DAG order or twice;
- every `local_swap`/`local_cx` acts on a genuine intra-core coupling edge, at the physical positions the verifier's own simulated layout says the operand logical qubits occupy;
- every `teleport` uses the correct inter-core port pair for its two cores, both ports were free beforehand, and the source qubit was coupling-adjacent to the source port;
- every `cat_entangler`, `cat_cx`, and `cat_disentangler` triple observes port-adjacency and free-port preconditions, and no other operation touches either communication port while a cat section is open;
- the emitted `RouterStats` counters (EPR, teleport, telegate, packet, SWAP, local-CX, single counts) match the counts the verifier itself derives from the op stream, catching any bookkeeping drift between what is emitted and what is counted.

We ran the verifier against 60 randomized circuits (4–40 logical qubits, 20–600 gates, both architectures, seeds 0–59) and against every circuit in the evaluation suite (Section IV of the main paper); all pass. Two further artefacts make these checks concrete: a *certified trace* (Figure 1) and a *fault-injection study* (Table III).

A certified trace. Figure 1 replays the stream PORTER emits for the running example of the main paper (Section III-D) — fourteen operations exercising every op type —

#	emitted operation		verifier asserts
0	`swap` $p_3 \leftrightarrow p_4$		coupling edge; operands sit where the replayed layout says
1	`cat_entangler` q_0; (p_5, p_{12}); 1 EPR		no section already open; *exact* port pair of (c_0, c_1); both ports free; q_0 adjacent to p_5; opens section, reserving p_5, p_{12}
2	`swap` $p_{10} \leftrightarrow p_9$	route q_1	tagged as packet routing; touches neither reserved port
3	`cat_cx` g_1 (p_{12}, p_9)		g_1 at the head of *both* q_0's and q_1's queues; control = section control; proxy is the target-side port p_{12}; target in c_1, adjacent to p_{12}
4	`swap` $p_{13} \leftrightarrow p_{14}$	route q_2	as op 2
5	`cat_cx` g_2 (p_{12}, p_{13})		as op 3
6	`cat_cx` g_3 (p_{12}, p_{15})		as op 3 (q_3 already port-adjacent)
7	`cat_disentangler`		a section is open; closes it, releasing the ports
8	`swap` $p_6 \leftrightarrow p_7$		coupling edge
9	`local_cx` g_5 (p_4, p_7)		queue heads; same core; coupling edge
10	`swap` $p_{13} \leftrightarrow p_{14}$		coupling edge
11	`teleport` q_5: $p_{13} \rightarrow p_5$; 1 EPR		cores adjacent; exact port pair (p_{12}, p_5); both ports free; q_5 adjacent to p_{12}; lands exactly on p_5
12	`swap` $p_4 \leftrightarrow p_5$		coupling edge
13	`local_cx` g_4 (p_3, p_4)		queue heads; coupling edge

Fig. 1. The full operation stream PORTER emits for the running example of the main paper (Section III-D, Figure 4), as replayed by the verifier. Ops 1–7 execute inside the live cat section, where only packet-tagged SWAPs and 1Q gates that avoid the two reserved ports are legal. At end of stream the verifier further asserts: no cat section left open, all five gates executed exactly once, and the emitted statistics equal to the derived counts (2 EPR, 1 teleport, 1 packet, 0 single telegates, 6 SWAPs, 2 local CXs).

TABLE III
FAULT-INJECTION VALIDATION OF THE VERIFIER (`VERIFY_FAULT_INJECTION.PY`): SINGLE-POINT CORRUPTIONS APPLIED TO OPERATION STREAMS THAT PASS VERIFICATION, EACH OF WHICH MUST BE REJECTED.

Injected corruption	Rejecting check
delete a `cat_disentangler`	section state machine
swap a teleport's two ports	exact inter-core port pair
zero a teleport's EPR cost	per-op EPR accounting
duplicate a gate-carrying op	every gate exactly once
delete a gate-carrying op	queue order / end-of-stream coverage
swap two ops sharing a qubit	per-qubit dependency order
retarget a SWAP to a non-coupled pair	coupling-edge legality
untag an in-section routing SWAP	cat-section isolation
retarget an in-section SWAP onto a port	reserved-port protection
increment `stats.epr` by one	stats–stream cross-check

and lists, per operation, exactly what the verifier asserts before accepting it. The trace makes the three primitives' legality conditions operational: the TELEGATEPACKET is one *bracketed section* in which the two reserved ports are untouchable and every interleaved SWAP must carry packet provenance; the TELEDATA at op 11 must name the unique port pair of its core pair, find both ports free, start port-adjacent, and land exactly on the destination port; and every gate execution, local or remote, is popped from the head of each operand's pending-gate queue, so DAG order is enforced gate by gate rather than assumed from the router's own bookkeeping.

Fault injection. On streams that always pass, a verifier that checks nothing is indistinguishable from one that checks everything, so we also certify that each check *fires*: `verify_fault_injection.py` takes passing op streams and applies the ten single-point corruptions of Table III, one mutant per corruption, spanning misuse of all three communication primitives (wrong ports, free-port violations, broken cat-section bracketing), gate-coverage and dependency-order violations, illegal SWAPs, and statistics drift. Across the running example (which admits all ten corruption classes) and thirty fuzzed circuits on both architectures, all 276 applicable mutants are rejected, each by the intended check; the same harness runs on any benchmark QASM file.

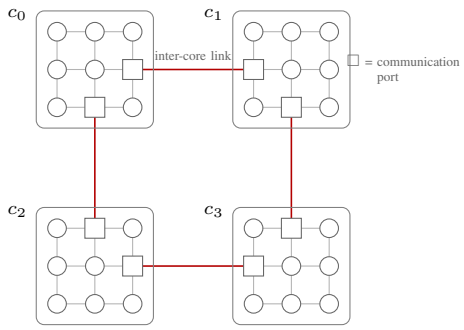


Fig. 2. Architecture A, introduced by TELESABRE [4]: a 2×2 array of cores, each a 3×3 grid of physical qubits (circles) with nearest-neighbour intra-core coupling (grey edges). Squares are communication ports; every inter-core link joins one port in each of two adjacent cores and is the only place EPR pairs are consumed. Architecture H (2×3 array of 4×4 cores, Section IV) follows the same pattern.

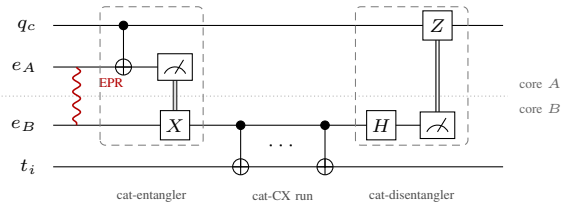


Fig. 3. The cat-entangler and cat-disentangler protocol [5] underlying TELEGATE and TELEGATEPACKET. The cat-entangler (left dashed box) consumes the EPR pair between ports e_A and e_B: $\mathrm{CX}(q_c, e_A)$, measurement of e_A, and a classically controlled X (double lines are classical bits) leave (q_c, e_B) in the cat state $\alpha|00\rangle + \beta|11\rangle$, making the remote port e_B a proxy for the control q_c. Every CX controlled by e_B (middle) then acts as if controlled by q_c — exactly one for TELEGATE, an ordered same-control run (interleaved with target-side 1Q gates) for TELEGATEPACKET. The cat-disentangler (right dashed box) — Hadamard, measurement, classically controlled Z — restores q_c and frees both ports.

REFERENCES

[1] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[2] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.

[3] S. Li, "dSABRE: A SABRE-style router for multi-core distributed quantum computers," 2026, manuscript, Centre for Quantum Software and Information, University of Technology Sydney. Code: https://github.com/ebony72/dsabre.

[4] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, "TeleSABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: https://github.com/Haimrich/telesabre.

[5] A. Yimsiriwattana and S. J. Lomonaco Jr., "Generalized GHZ states and distributed quantum computing," *arXiv preprint quant-ph/0402148*, 2004.

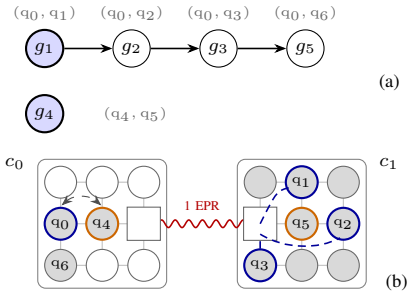


Fig. 4. Running example. (a) Remaining circuit fragment as a DAG; the front (shaded) is $\{g_1, g_4\}$. (b) Layout on two adjacent cores of architecture A. Blue circles are the committed TELEGATEPACKET's participants (q_0 and the targets q_1, q_2, q_3, reached by cat-CXs, dashed); orange circles are the endpoints of the competing, already-ready single TELEGATE for g_4; grey circles are spectators. Core c_1's only free position is its port, so $\text{free}(c_1) = 1 < f_{\min}$ and every TELEDATA into c_1 is capacity-blocked. The dashed double arrow is the packet's one preparation SWAP, which moves q_0 next to the port.

TABLE IV
REMOTE PLANS GENERATED FOR THE FRONT OF FIGURE 4, WITH TOTAL GAINS (SECTION III-E).

Plan	Set	Gain	Readiness	Outcome
TELEGATEPACKET(q_0; $g_1 g_2 g_3$)	$\mathcal{P}$	3.0	1 prep SWAP	**committed**
TELEGATE(g_4)	$\mathcal{P}$	1.0	ready now	dominated
TELEGATE(g_1)	$\mathcal{P}$	1.0	1 prep SWAP	dominated
TELEDATA($q_1 \to c_0$)	$\mathcal{P}$	1.0	prep needed	dominated
TELEDATA($q_5 \to c_0$)	$\mathcal{P}$	1.0	prep needed	committed later
TELEDATA($q_0 \to c_1$)	$\mathcal{B}$	≈ 2.1	blocked	margin $< m_{\text{assist}}$
TELEDATA($q_4 \to c_1$)	$\mathcal{B}$	1.0	blocked	dominated

APPENDIX E
ARCHITECTURE AND PRIMITIVE FIGURES

APPENDIX F
RUNNING EXAMPLE: WHY COMMIT TO A PLAN?

Figure 4 shows a front where plan commitment, the packet vocabulary, and the $f_{\min}$ capacity gate all interact, and where PORTER's decision differs from what either baseline would choose. Two adjacent cores of architecture A are shown. Core c_0 holds q_0, q_4 (adjacent to the port), and q_6, with six free positions; core c_1 holds eight qubits — the targets q_1, q_2, q_3, the port-adjacent q_5, and four spectators — so its *only* free position is its port. The remaining circuit fragment is $g_1{=}\text{CX}(q_0, q_1)$, $g_2{=}\text{CX}(q_0, q_2)$, $g_3{=}\text{CX}(q_0, q_3)$, $g_5{=}\text{CX}(q_0, q_6)$, chained on q_0, plus an independent $g_4{=}\text{CX}(q_4, q_5)$. The front is therefore $\{g_1, g_4\}$ — g_2 and g_3 share q_0 with g_1 and only surface after it executes. All front gates are adjacent-remote, so the plan-first stage fires.

What PORTER does. Table IV lists the generated plans. Seeding packet collection at g_1, the frontier simulation absorbs g_2 and g_3 (same control q_0, targets in c_1): a three-gate packet with direct gain 3. Both single telegates and all TELEDATA plans into c_0 gain ≈ 1. TELEDATA($q_0 \to c_1$) would gain more — direct 1 for g_1, latent-direct 2 for g_2, g_3, minus 0.9 of lookahead for g_5, whose partner q_6 stays in c_0 — but c_1 fails the $f_{\min}$ test, so it lands in $\mathcal{B}$; its gain ≈ 2.1 does not exceed the packet's 3.0 by $m_{\text{assist}}{=}5$, so no capacity relief pre-empts. Plans are ranked by gain first — readiness is only a tie-break — so the *unready* packet (gain 3) beats the *ready* telegate for g_4 (gain 1). PORTER commits the packet, generates its one preparation SWAP (q_0 next to the port), then emits one cat-entangler, three cat-CXs (each target routed beside the port in turn), and the disentangler: *one* EPR pair for g_1, g_2, g_3. The packet's completion exposes g_4 and g_5 together; the locality-first stage drains the now-local g_5 first (one SWAP plus a local CX), and the remote stage then re-generates plans for g_4: the single telegate and TELEDATA($q_5 \to c_0$) now tie at gain 1, and the plan-kind tie-break (TELEDATA $\succ$ TELEGATE, Section III-E) commits the teleport — q_5 lands on c_0's port and g_4 finishes as a local CX. Two EPR pairs for the whole fragment. This is not a stylised trace: it is exactly the operation stream the implementation emits on this instance, replayed and certified by the independent verifier of Appendix D, which includes the full operation-by-operation certified trace.

What finer-grained routers do here. A router that scores individual actions — SWAPs and inter-core operations in one pool, as TELESABRE does — fires the zero-cost ready telegate for g_4 first. The deeper problem is the staging SWAP the packet needs: exchanging q_0 and q_4 moves g_1's control one step closer to the port while moving g_4's one step away, so under a front-averaged distance metric of the kind used by SABRE descendants its score is ≈ 0; the information that makes this SWAP worth applying — three remote CXs behind g_1 amortised onto one EPR pair — is a property of a *whole plan*, invisible at single-action granularity. That blind spot does not require a flat pool: DSABRE reaches its inter-core stage on this front (no front gate is intra-core) and scores hops, not plans, so the same amortisation is outside what its candidate can express.

Vocabulary compounds the gap. Gates g_2 and g_3 enter the front one at a time, so TELESABRE pays one telegate each, four EPR pairs for the fragment. DSABRE, restricted to TELEDATA, cannot move q_0 into the saturated c_1 at all — under its capacity floor a teleport into a core with fewer than two free slots is not a legal candidate — so it must first relay a free slot into c_1 from elsewhere, and ferrying q_1, q_2, q_3 out to c_0 instead costs three EPR pairs before g_4 is even addressed. Commitment finally protects the choice once made: only the packet's preparation SWAPs are generated — no staging effort is spent on the six losing plans — and the committed plan cannot be pre-empted mid-preparation by a candidate that its own staging transiently improved, a churn per-action re-scoring permits by construction.

APPENDIX G
INTER-CORE ACTION MIX

Table V puts the three compilers' primitives side by side on the 64-qubit suite, which is the sharpest statement of what a larger action vocabulary is worth: every action listed costs exactly one EPR pair, so an action count and an EPR count are the same number — except for PORTER's packets, where one action executes several gates.

TELESABRE spends 289 of its 4,478 actions (6.5%) on single telegates, each serving exactly one remote CX, and uses the most EPR pairs of the three. DSABRE spends none. PORTER's cat-entangler actions are 1,438 of 4,500 (32.0%), but they are not the same object: 801 packets carry 1,976

TABLE V
INTER-CORE ACTIONS BY PRIMITIVE ON THE 64-QUBIT SUITE. EVERY ACTION CONSUMES EXACTLY ONE EPR PAIR, SO THE COLUMNS OF EACH ROUTER SUM TO ITS EPR COUNT. "PKT CX" IS THE NUMBER OF REMOTE CX GATES PORTER'S PACKETS EXECUTE — THE ONLY COLUMN IN WHICH AN ACTION COUNT AND A GATE COUNT DIFFER. TELESABRE DOES NOT CONVERGE ON RANDOM.

	TELESABRE		DSABRE	PORTER			
Circuit	TELEDATA	TELEGATE	TELEDATA	TELEDATA	TELEGATE	pkt	pkt CX
AE	259	94	165	145	0	32	101
GHZ	16	0	6	6	0	0	0
Graphstate	60	2	15	20	0	0	0
QFT	308	4	179	216	1	45	105
QNN	883	6	538	619	5	240	630
Random	—	—	732	432	228	77	157
QPEexact	281	8	187	177	3	42	130
QAOA	824	132	555	507	16	331	761
Multiplier	1558	43	1285	940	384	34	92
Total	**4189**	**289**	**3662**	**3062**	**637**	**801**	**1976**

remote CX gates, 2.47 per EPR pair, against 1.00 for the 637 single telegates. Packets are 56% of PORTER's cat-entangler actions and 76% of the gates those actions serve.

Two features of the table shape the ablation of Section V-C. First, the vocabulary is used very unevenly: Random and Multiplier account for 612 of PORTER's 637 single telegates, while QNN and QAOA account for 571 of its 801 packets. Second, amortisation and EPR consumption do not move together — PORTER spends more cat-entangler actions than TELESABRE and fewer EPR pairs than it, but also more EPR pairs than DSABRE, which spends none at all. Whether the extra vocabulary pays is therefore not answerable from this table; Section V-C answers it directly, and the answer is not the one the design assumed.

APPENDIX H
SUPPLEMENTARY TABLES

The tables below support claims made in Sections III and V of the main paper: the full parameter set (Table VI), the per-circuit heavy-hex and shared-layout rows summarised there (Tables VII and VIII), and local SWAP counts (Table X). EPR pairs are the figure of merit throughout — a SWAP is a local operation costing no entanglement — which is why SWAP counts are reported here rather than in the main text.

Table IX gives the per-circuit EPR counts summarised by Table II of the main paper.

Table X gives the intra-core SWAP counts for the runs of Table V of the main paper, in the same row order. Both baselines and PORTER count intra-core SWAPs only; inter-core movement is charged in EPR pairs.

APPENDIX I
COMPUTATIONAL COMPLEXITY

Let N be the gate count, n the logical-qubit count, K the core count and M the per-core physical-qubit count. Distance tables are precomputed once at construction in $O(KM^2 \log M + K^2 \log K)$, matching the two-level Dijkstra decomposition comparable compilers use [3]. Per iteration, classification and draining cost $O(|F|)$; intra-core SWAP scoring evaluates $O(M)$ candidate edges, each at $O(|F_{\text{loc}}| + N_{\text{loc}})$ over one DAG snapshot reused across candidates; remote-plan scoring evaluates $O(n)$ candidates at $O(N_{\text{tp}})$ over the same snapshot. Free-slot queues are $O(1)$ amortised per layout mutation. Summing over N iterations gives $T_{\text{total}} = O(N(Mn + KM^2 \log M))$ excluding precomputation, in line with other core-local distributed compilers [3], [4].

TABLE VI
HARDWARE COST AND HEURISTIC PARAMETERS (SEE SECTION III).

Parameter	Symbol	Value
Min. free before TELEDATA	$f_{\min}$	2
Opportunistic-relief margin	m_{assist}	5.0
Local-SWAP extended-set weight	λ_{loc}	0.8
Local extended-set max layers	L_{loc}	12
Local extended-set max CX	N_{loc}	20
Packet collection window (layers)	–	120
Max packet size (remote CX)	–	8
TeleData lookahead max layers	L_{tp}	$\{4, 8, 16\}$ or $\{8, 16, 128\}^{\dagger}$
TeleData lookahead max CX	N_{tp}	$50^{\ddagger}$
Lookahead weight / decay / floor	$w_0, \gamma, w_{\min}$	0.9, 0.9, 0.5
DAG front cap (gates)	–	96

† The teleport-lookahead horizon is not a fixed constant: PORTER routes the circuit once per candidate value and keeps the lowest-EPR result, taking the short candidate set when the interaction graph is near all-to-all and the long one otherwise (Section IV-E1).
‡ Inert at this value. The lookahead admits at most one gate on the moving qubit per simulated layer, so the collected count cannot exceed L_{tp} and the cap cannot bind while $N_{\text{tp}} \geq L_{\text{tp}}$; over 3 830–11 395 scoring calls per circuit it was never reached, and changing it leaves the emitted circuit bit-for-bit identical.

APPENDIX J
THE LAYOUT-DISCOUNT SEARCH (PORTER+S)

PORTER+S is a third operating protocol, reported here rather than in the main paper because it is a parameter search rather than a design decision, and because what it returns does not justify what it costs. It searches the one free scalar in the initial-layout heuristic — the per-layer discount γ of Section III-A, fixed at 0.99 everywhere else — in two phases.

Phase 1 builds a layout for each $\gamma \in \{0.80, 0.90, 0.95, 0.99, 1.0\}$ and routes it once at the configured horizon — five routes — keeping the lowest-EPR layout. Phase 2 runs the P=5 backward-refinement schedule of Section IV-E2 from two seeds: phase 1's winner, and the γ=0.99 layout when that is not already the winner, adding $P-1$=4 routes each. Refining the phase-1 winner alone is not reliably best, so both are run. The total is $5 + 4 = 9$ routes when γ=0.99 wins and $5 + 4 + 4 = 13$ otherwise, plus the $|\mathcal{H}| - 1$ extra probes each seed's horizon selection spends: 10 to 17 routes. Since γ=0.99 always competes and is refined from its selected route, PORTER+S cannot return a higher EPR count than PORTER or PORTER+R do.

No single γ dominates — all five win somewhere, 0.80 most often (nine of twenty-four instances) — which is why the value is searched rather than retuned. The search is deterministic and takes no seed.

What it buys. Table XI gives the result. Pooled over the twenty-four instances PORTER+S reaches 159.1 EPR pairs against PORTER+R's 163.0, a 2.4% reduction, and 0.93× DSABRE's 171.9 against PORTER+R's 0.95×. It improves on ten instances, ties on fourteen and — being a superset

TABLE VII

ARCHITECTURE-INDEPENDENCE CHECK: THE SAME NINE-CIRCUIT 64-QUBIT SUITE ON FOUR IBM 27-QUBIT HEAVY-HEX CORES (108 PHYSICAL QUBITS), WIRED AS A *ring* (4 LINKS, EVERY CORE DEGREE 2) AND AS A *star* (3 LINKS, CORE 0 A HUB EVERY INTER-CORE ROUTE MUST CROSS). EPR PAIRS ONLY; THE QUESTION HERE IS ARCHITECTURAL SENSITIVITY, NOT THE EPR/SWAP TRADE-OFF OF TABLE II. THE RIGHT-HAND PAIR IS THE FIXED-LAYOUT CONTROL: PORTER ROUTING DSABRE'S OWN SABRELAYOUT CANDIDATES, SO THE TWO ROUTERS DIFFER ONLY IN THE ROUTING LOOP. REFINEMENT IS MEASURED ON THE MAIN SUITE ONLY (TABLE II), SO NO PORTER+R COLUMN APPEARS HERE. DSABRE REPRODUCES ITS OWN PUBLISHED NUMBERS FOR THESE TWO ARCHITECTURES ON ALL EIGHTEEN INSTANCES [3].

			own layout	DSABRE layout
Cores	Circuit	DSABRE	PORTER	PORTER
ring	ae	86	69	61
ring	ghz	2	2	2
ring	graphstate	12	12	13
ring	qft	110	109	110
ring	qnn	249	240	269
ring	random	465	465	419
ring	qpeexact	80	105	93
ring	qaoa	249	244	244
ring	multiplier	451	506	416
gmean (9)		85.2	86.1	**82.9**
star	ae	118	149	104
star	ghz	3	3	4
star	graphstate	10	16	11
star	qft	125	117	108
star	qnn	241	449	321
star	random	475	547	478
star	qpeexact	120	162	102
star	qaoa	251	405	245
star	multiplier	722	455	462
gmean (9)		101.1	122.3	**98.5**

TABLE VIII

SHARED-LAYOUT COMPARISON, ALL SUITES: DSABRE AND PORTER ARE BOTH ROUTED FROM TELESABRE'S OWN INITIAL LAYOUT, EACH AT ITS OWN ROUTE BUDGET (SECTION IV-D). Δ_{TS} COLUMNS GIVE EACH ROUTER'S EPR CHANGE RELATIVE TO TELESABRE'S OWN-PROTOCOL RESULT; NEGATIVE IS BETTER. ROWS EXIST ONLY WHERE TELESABRE CONVERGES AND THUS PROVIDES A LAYOUT TO SHARE: ALL SIX CIRCUITS AT 25Q, EIGHT OF NINE AT 64Q (RANDOM MISSING), THREE OF NINE AT 80Q (GHZ, GRAPHSTATE, AND QPEEXACT); PER-SUITE GMEANS ARE OVER THE AVAILABLE ROWS, AND THE FINAL **ALL** ROW IS THE GMEAN OVER ALL 17.

		TELESABRE		DSABRE			PORTER		
Suite	Circuit	EPR	SWAP	EPR	SWAP	Δ_{TS}	EPR	EPR_{+R}	Δ_{TS}
25q	ae	119	346	66	310	−44.5	78	60	−49.6
25q	ghz	13	44	9	29	−30.8	10	8	−38.5
25q	graphstate	21	63	6	30	−71.4	14	6	−71.4
25q	qft	138	448	81	362	−41.3	100	97	−29.7
25q	qnn	515	820	145	685	−71.8	136	126	−75.5
25q	random	636	1367	419	1442	−34.1	397	367	−42.3
gmean (6)		106.6		51.0		**−52.2**	62.4	48.4	**−54.6**
64q	ae	353	1494	215	1672	−39.1	151	151	−57.2
64q	ghz	16	132	10	121	−37.5	10	8	−50.0
64q	graphstate	62	281	26	149	−58.1	46	24	−61.3
64q	qft	312	1711	223	1646	−28.5	290	257	−17.6
64q	qnn	889	5471	648	5707	−27.1	580	580	−34.8
64q	qpeexact	289	1706	193	1574	−33.2	186	186	−35.6
64q	qaoa	956	4189	673	4244	−29.6	598	598	−37.4
64q	multiplier	1601	9654	1564	9489	−2.3	873	755	−52.8
gmean (8)		284.5		189.2		**−33.5**	180.7	156.7	**−44.9**
80q	ghz	26	215	25	196	−3.8	19	12	−53.8
80q	graphstate	87	435	40	233	−54.0	64	32	−63.2
80q	qpeexact	784	2875	408	2732	−48.0	436	436	−44.4
gmean (3)		121.0		74.2		**−38.7**	80.9	55.1	**−54.5**
gmean, all (17)		173.0		100.9		**−41.7**	107.7	**86.1**	**−50.2**

schedule — loses on none. Its one qualitative effect is at 64 qubits, the single suite PORTER loses under its own protocol: the ratio against DSABRE falls from PORTER+R's 1.07× to 1.02×.

What it costs. The same twenty-four instances take 4,267 s against PORTER+R's 1,468 s — 2.9× the wall clock, and 23× DSABRE's complete protocol cost of 182 s. The expenditure is badly distributed: the 80-qubit suite consumes 4,045 of those seconds and returns 1.2%, and 80q AE alone spends 928 s against PORTER+R's 14.9 s — a factor of 62 — to move from 299 pairs to 284. On more than half the suite the search returns nothing at all.

Why it is not in the main paper. Three reasons beyond the return. It cannot appear in the layout-against-router grid of Table III, since its mechanism *is* choosing the initial layout and a supplied layout would destroy that control — so it can only ever be a fourth own-protocol column. It introduces a second in-sample-fitted quantity, the γ set, alongside the horizon candidate sets whose in-sample fitting Section IV-E1 already declares. And at 10–17 routes and 6.55× DSABRE's compile time it sits on the wrong side of the argument Section V-F makes, that PORTER's time advantage lies in the number of routes it spends rather than their individual price. We report it here so the shape of the EPR/time frontier past PORTER+R is on the record.

PORTER+SR is subsumed. An earlier version of this work also reported PORTER+SR, the better of horizon selection and the γ-search. Phase 2 refines from the γ=0.99 layout among its seeds, and that is the layout PORTER builds by default, so once refinement runs on top of horizon selection the search contains the selected route and cannot lose to it. PORTER+SR does not differ from PORTER+S on any of the twenty-four instances, at 19% more compile time, and is retired.

TABLE IX

PER-CIRCUIT EPR COUNTS, OWN PROTOCOL — THE DETAIL BEHIND TABLE II OF THE MAIN PAPER. TELESABRE AND DSABRE REPORT THE BEST OF THREE SEEDS; PORTER ROUTES ONE DETERMINISTIC LAYOUT AT EACH OF THREE CANDIDATE LOOKAHEAD HORIZONS; AND PORTER+R SPENDS FIVE ROUTES ON BACKWARD REFINEMENT INSTEAD. Δ_{TS} IS THE EPR CHANGE OF WHICHEVER PORTER COLUMN IS STRONGER ON THAT ROW, AGAINST TELESABRE (%); NEGATIVE IS BETTER, AND THE BOLDED PORTER COLUMN IS THAT STRONGER ONE. *TELESABRE DOES NOT CONVERGE IN THREE SEEDS; EACH GMEAN ROW COVERS THE CIRCUITS IT COMPLETES. LOCAL SWAP COUNTS, IN THE SAME ROW ORDER, ARE IN TABLE X.

Suite	Circuit	TELESABRE	DSABRE	PORTER	PORTER+R	Δ_{TS}
25q	ae	119	65	65	65	−45.4
25q	ghz	13	5	4	4	−69.2
25q	graphstate	21	6	7	4	−81.0
25q	qft	138	81	74	74	−46.4
25q	qnn	515	143	119	116	−77.5
25q	random	636	411	401	383	−39.8
gmean (6)		106.6	45.8	43.1	**38.8**	**−63.6**
64q	ae	353	165	157	157	−55.5
64q	ghz	16	6	6	6	−62.5
64q	graphstate	62	15	20	16	−74.2
64q	qft	312	179	262	262	−16.0
64q	qnn	889	538	638	638	−28.2
64q	random	*	732	737	737	—
64q	qpeexact	289	187	222	222	−23.2
64q	qaoa	956	555	775	586	−38.7
64q	multiplier	1601	1285	1076	1076	−32.8
gmean (8)		284.5	144.6	166.2	**156.1**	**−45.1**
80q	ae	*	332	299	299	—
80q	ghz	26	13	7	7	−73.1
80q	graphstate	87	20	28	24	−72.4
80q	qft	*	322	446	393	—
80q	qnn	*	807	1070	918	—
80q	random	*	10551	11040	11040	—
80q	qpeexact	784	347	291	291	−62.9
80q	qaoa	*	862	1122	1122	—
80q	multiplier	*	4841	2411	2132	—
gmean (3)		121.0	44.9	38.5	**36.6**	**−69.8**

TABLE X

LOCAL SWAP COUNTS, OWN PROTOCOL. ROW ORDER AND THE * NON-CONVERGENCE MARKER MATCH TABLE V OF THE MAIN PAPER.

Suite	Circuit	TELESABRE	DSABRE	PORTER	PORTER+R
25q	ae	346	309	328	328
25q	ghz	44	21	29	29
25q	graphstate	63	31	34	19
25q	qft	448	365	327	327
25q	qnn	820	711	642	603
25q	random	1367	1390	1327	1319
64q	ae	1494	1325	1627	1627
64q	ghz	132	39	86	86
64q	graphstate	281	88	131	103
64q	qft	1711	1480	1846	1846
64q	qnn	5471	5260	6595	5608
64q	random	*	3603	4048	4048
64q	qpeexact	1706	1576	1729	1729
64q	qaoa	4189	3734	5394	4256
64q	multiplier	9654	8969	9878	8264
80q	ae	*	2624	3156	3156
80q	ghz	215	51	115	115
80q	graphstate	435	97	198	150
80q	qft	*	2311	2968	2692
80q	qnn	*	8590	10920	9959
80q	random	*	56624	62119	62119
80q	qpeexact	2875	2254	3427	3427
80q	qaoa	*	6067	7370	7370
80q	multiplier	*	19196	18863	16810

TABLE XI

PORTER+S AGAINST PORTER AND PORTER+R ON THE MAIN TWENTY-FOUR INSTANCES, OWN PROTOCOL. γ^* IS THE DISCOUNT PHASE 1 SELECTED; t IS COMPLETE PROTOCOL TIME IN SECONDS. Δ IS PORTER+S AGAINST PORTER+R AND IS DASHED WHERE THE TWO COINCIDE. PORTER+S IS NEVER WORSE, BY CONSTRUCTION. LOWER IS BETTER.

		EPR					t (s)	
Suite	Circuit	PORTER	+R	+S	Δ %	γ^*	+R	+S
25q	ae	65	65	62	−4.6	0.80	0.5	1.4
25q	ghz	4	4	4	—	0.80	0.0	0.0
25q	graphstate	7	4	4	—	0.80	0.0	0.1
25q	qft	74	74	74	—	0.99	0.6	1.0
25q	qnn	119	116	116	—	0.80	1.2	3.2
25q	random	401	383	379	−1.0	0.80	2.7	7.7
gmean (6)		43.1	38.8	**38.4**	**−1.0**		0.26	0.65
64q	ae	157	157	149	−5.1	0.95	8.4	16.3
64q	ghz	6	6	6	—	0.80	0.0	0.1
64q	graphstate	20	16	16	—	0.80	0.3	0.9
64q	qft	262	262	262	—	0.99	7.4	9.8
64q	qnn	638	638	604	−5.3	1.00	10.4	36.0
64q	random	737	737	664	−9.9	0.90	21.0	53.3
64q	qpeexact	222	222	197	−11.3	1.00	7.4	13.2
64q	qaoa	775	586	586	—	0.95	18.5	44.6
64q	multiplier	1076	1076	998	−7.2	0.90	14.0	34.6
gmean (9)		196.1	185.5	**177.3**	**−4.4**		4.00	9.31
80q	ae	299	299	284	−5.0	0.90	14.9	928.0
80q	ghz	7	7	7	—	0.80	0.0	0.1
80q	graphstate	28	24	24	—	0.80	0.6	1.8
80q	qft	446	393	377	−4.1	1.00	8.4	18.9
80q	qnn	1070	918	918	—	1.00	19.0	68.9
80q	random	11040	11040	10836	−1.8	1.00	357.5	1957.6
80q	qpeexact	291	291	291	—	0.95	908.7	943.1
80q	qaoa	1122	1122	1122	—	0.99	41.4	62.4
80q	multiplier	2411	2132	2132	—	0.95	25.2	64.1
gmean (9)		396.7	372.9	**368.3**	**−1.2**		14.04	50.23
gmean, all (24)		174.9	163.0	**159.1**	**−2.4**		3.22	9.01